

beginning

```
from google.colab import drive
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

data loading

```
df_org = pd.read_excel("/content/drive/MyDrive/capstone/datasets/0_original.xlsx")
df_org.head()
```

	data_source	full_name	age	age_group	family_members	position_in_siblings	gender	maritual_status	duration_of_separataion	profession_group
0	https://www.dhakatribune.com/bangladesh/nation...	Parvez	NaN	Teen	NaN	NaN	Male	NaN	NaN	Worke
1	https://www.dhakatribune.com/bangladesh/nation...	Alhaj Uddin	20.0	Teen	NaN	NaN	Male	relationship-rejected	NaN	NaN
2	https://www.dhakatribune.com/bangladesh/nation...	Priti Akter	14.0	Teen	NaN	NaN	Female	unmarried	NaN	Studen
3	https://www.dhakatribune.com/bangladesh/nation...	Acchia Akhter	NaN	NaN	NaN	NaN	Female	relationship-rejected	NaN	Studen
4	https://www.dhakatribune.com/bangladesh/dhaka/...	Dilshad Nahar	18.0	Teen	NaN	NaN	Female	relationship	NaN	Studen

5 rows × 36 columns

```
df_clr = pd.read_excel("/content/drive/MyDrive/capstone/datasets/1_cleaned.xlsx")
df_clr.head()
```

	age	age_group	gender	marital_status	profession_group	profession_description	workplace	religion	financial_condition	hometown	...	feels_like	temp_min	temp_max
0	NaN	teen	male	NaN	worker	he was an plumber	rural	muslim	indebted	brahmanbaria	...	302.86	299.15	299.15
1	20.0	teen	male	rejected	NaN	NaN	NaN	muslim	NaN	sylhet	...	299.17	295.57	295.57
2	14.0	teen	female	single	student	student of class nine	urban	muslim	NaN	manikganj	...	304.33	301.15	301.15
3	NaN	NaN	female	rejected	student	student of Bangla Department	urban	muslim	NaN	bogura	...	303.67	299.34	299.34
4	18.0	teen	female	relationship	student	studied fashion design at a private university	urban	muslim	NaN	dhaka	...	304.49	300.15	300.15

5 rows × 28 columns

```
df_lat_lon = pd.read_excel("/content/drive/MyDrive/capstone/datasets/2_lat_lon.xlsx")
df_lat_lon.head()
```

	age	age_group	gender	marital_status	profession_group	profession_description	workplace	religion	financial_condition	hometown	...	feels_like	temp_min	temp_max
0	NaN	teen	male	NaN	worker	he was an plumber	rural	muslim	indebted	brahmanbaria	...	302.86	299.15	299.15
1	20.0	teen	male	rejected	NaN	NaN	NaN	muslim	NaN	sylhet	...	299.17	295.57	295.57
2	14.0	teen	female	single	student	student of class nine	urban	muslim	NaN	manikganj	...	304.33	301.15	301.15
3	NaN	NaN	female	rejected	student	student of Bangla Department	urban	muslim	NaN	bogura	...	303.67	299.34	299.34
4	18.0	teen	female	relationship	student	studied fashion design at a private university	urban	muslim	NaN	dhaka	...	304.49	300.15	300.15

5 rows × 28 columns

```
df_lat_lon.columns
```

```
Index(['age', 'age_group', 'gender', 'marital_status', 'profession_group',
      'profession_description', 'workplace', 'religion',
      'financial_condition', 'hometown', 'latitude', 'longitude', 'reason',
      'reason_description', 'time', 'method', 'suicide_date', 'unix_time',
      'feels_like', 'temp_min', 'temp_max', 'air_pressure', 'air_humidity',
      'wind_speed', 'wind_deg', 'clouds_sky', 'weather_main',
      'weather_description'],
      dtype='object')
```

```
def missing_data_report(df):
    # Calculate the percentage of missing data for each column
    missing_percentage = df.isna().mean() * 100

    # Create a DataFrame for the missing data summary
    missing_data_summary = pd.DataFrame({
        'Column': missing_percentage.index,
        'Missing Percentage': missing_percentage.values
    })
```

```
# Define the four groups based on missing percentage
group_1 = missing_data_summary[missing_data_summary['Missing Percentage'] > 70][['Column', 'Missing Percentage']]
group_2 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 40) & (missing_data_summary['Missing Percentage'] <= 70)][['Column', 'Missing Percentage']]
group_3 = missing_data_summary[(missing_data_summary['Missing Percentage'] > 20) & (missing_data_summary['Missing Percentage'] <= 40)][['Column', 'Missing Percentage']]
group_4 = missing_data_summary[missing_data_summary['Missing Percentage'] <= 20][['Column', 'Missing Percentage']]

# Sort each group by missing percentage in descending order (most missing to least missing)
group_1 = group_1.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
group_2 = group_2.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
group_3 = group_3.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)
group_4 = group_4.sort_values(by='Missing Percentage', ascending=False).reset_index(drop=True)

# Rename columns for clarity
group_1.columns = ['Column', 'Missing %']
group_2.columns = ['Column', 'Missing %']
group_3.columns = ['Column', 'Missing %']
group_4.columns = ['Column', 'Missing %']

# Display each group one by one, from most missing to least missing
print("Group 1: Columns with >70% missing data")
print(group_1.to_string(index=False))
print("\nGroup 2: Columns with >40% to ≤70% missing data")
print(group_2.to_string(index=False))
print("\nGroup 3: Columns with >20% to ≤40% missing data")
print(group_3.to_string(index=False))
print("\nGroup 4: Columns with ≤20% missing data")
print(group_4.to_string(index=False))
```

```
missing_data_report(df_lat_lon)
```

```
Group 1: Columns with >70% missing data
Empty DataFrame
Columns: [Column, Missing %]
Index: []
```

```
Group 2: Columns with >40% to ≤70% missing data
      Column  Missing %
temp_min    53.061224
feels_like  53.061224
air_humidity 53.061224
air_pressure 53.061224
temp_max    53.061224
clouds_sky   53.061224
wind_deg     53.061224
weather_main 52.999382
wind_speed   52.999382
weather_description 52.999382
unix_time    52.380952
profession_description 49.412492
financial_condition 45.516388
```

```
Group 3: Columns with >20% to ≤40% missing data
      Column  Missing %
workplace    37.909709
marital_status 23.933210
```

```
Group 4: Columns with ≤20% missing data
```

Column	Missing %
profession_group	18.923933
reason	16.821274
reason_description	16.635745
age	12.739641
time	12.059369
age_group	7.421150
latitude	6.493506
longitude	6.493506
method	2.906617
religion	2.411874
suicide_date	2.040816
hometown	1.669759
gender	0.432900

```
categorical_columns = [
    'age_group', 'gender', 'religion', 'marital_status', 'profession_group', 'workplace',
    'financial_condition', 'reason', 'time', 'method', 'hometown', 'weather_main', 'weather_description',
]

descriptive_columns = ['profession_description', 'reason_description']

numerical_columns = [
    'age', 'latitude', 'longitude', 'unix_time', 'feels_like', 'temp_min', 'unix_time',
    'temp_max', 'air_pressure', 'air_humidity', 'wind_speed', 'wind_deg', 'clouds_sky'
]

weather_columns = [
    'feels_like', 'temp_min', 'temp_max', 'air_pressure',
    'air_humidity', 'wind_speed', 'wind_deg', 'clouds_sky',
    'weather_main', 'weather_description',
]

# only date format in ths database is "suicide_date"
```

```
import pandas as pd
import numpy as np
from datetime import datetime

# Mapping of time of day to representative clock time
time_to_hour = {
    'morning': '08:00',
    'noon': '12:00',
    'afternoon': '15:00',
    'evening': '18:00',
    'night': '21:00',
    'day': '10:00'
}

# Step 1: Create a new column 'time_hour' from 'time' using the mapping
df_lat_lon['time_hour'] = df_lat_lon['time'].map(time_to_hour)

# Step 2: Define function to estimate unix_time from date and time_hour
def estimate_unix(row):
    date_str = row['suicide_date']
    time_str = row['time_hour']
```

```

if pd.isna(date_str) or pd.isna(time_str):
    return row['unix_time'] # leave as-is

try:
    dt = datetime.strptime(f"{date_str} {time_str}", "%d/%m/%Y %H:%M")
    return int(dt.timestamp())
except Exception:
    return row['unix_time'] # leave as-is if parsing fails

```

```

df_lat_lon['unix_time'] = df_lat_lon.apply(
    lambda row: estimate_unix(row) if pd.isnull(row['unix_time']) else row['unix_time'],
    axis=1
)

```

```
df_lat_lon.columns
```

```

Index(['age', 'age_group', 'gender', 'marital_status', 'profession_group',
       'profession_description', 'workplace', 'religion',
       'financial_condition', 'hometown', 'latitude', 'longitude', 'reason',
       'reason_description', 'time', 'method', 'suicide_date', 'unix_time',
       'feels_like', 'temp_min', 'temp_max', 'air_pressure', 'air_humidity',
       'wind_speed', 'wind_deg', 'clouds_sky', 'weather_main',
       'weather_description', 'time_hour'],
      dtype='object')

```

```
missing_data_report(df_lat_lon)
```

```

Group 1: Columns with >70% missing data
Empty DataFrame
Columns: [Column, Missing %]
Index: []

```

```

Group 2: Columns with >40% to ≤70% missing data
      Column  Missing %
temp_min    53.061224
feels_like  53.061224
air_pressure 53.061224
temp_max    53.061224
clouds_sky  53.061224
wind_deg    53.061224
air_humidity 53.061224
wind_speed  52.999382
weather_main 52.999382
weather_description 52.999382
profession_description 49.412492
financial_condition 45.516388

```

```

Group 3: Columns with >20% to ≤40% missing data
      Column  Missing %
workplace   37.909709
marital_status 23.933210

```

```

Group 4: Columns with ≤20% missing data
      Column  Missing %
profession_group 18.923933
reason          16.821274
reason_description 16.635745

```

```
age 12.739641
time 12.059369
time_hour 12.059369
age_group 7.421150
longitude 6.493506
latitude 6.493506
unix_time 5.009276
method 2.906617
religion 2.411874
suicide_date 2.040816
hometown 1.669759
gender 0.432900
```

```
!pip install meteostat tqdm pytz --quiet
```

```
import pandas as pd
from meteostat import Point, Hourly, Daily
from datetime import datetime, timedelta
from tqdm import tqdm
import pytz
```

```
dhaka_tz = pytz.timezone("Asia/Dhaka")

def unix_to_dt(unix_time):
    try:
        utc_dt = datetime.utcfromtimestamp(unix_time)
        utc_dt = pytz.UTC.localize(utc_dt)
        dhaka_dt = utc_dt.astimezone(dhaka_tz)
        return dhaka_dt
    except Exception:
        return pd.NaT

# Weather columns to fill
weather_cols = ['temp_min', 'temp_max', 'air_pressure', 'air_humidity',
                'wind_speed', 'wind_deg', 'coco']
```

```
for col in weather_cols:
    if col not in df_lat_lon.columns:
        df_lat_lon[col] = pd.NA
```

```
def fill_weather(row):
    if all(pd.isna(row[col]) for col in ['temp_min', 'temp_max', 'air_pressure', 'air_humidity', 'wind_speed', 'wind_deg']):
        return row

    lat, lon, unix_time = row['latitude'], row['longitude'], row['unix_time']

    if pd.isna(lat) or pd.isna(lon) or pd.isna(unix_time):
        return row

    dt = unix_to_dt(unix_time)
    if pd.isna(dt):
        return row

    point = Point(lat, lon)
```

```

# Try hourly data first
try:
    from_dt = dt - timedelta(hours=1)
    to_dt = dt + timedelta(hours=1)
    hourly = Hourly(point, from_dt, to_dt)
    hourly_data = hourly.fetch()

    if not hourly_data.empty:
        if hourly_data.index.tz is None:
            hourly_data.index = hourly_data.index.tz_localize('UTC').tz_convert('Asia/Dhaka')
        else:
            hourly_data = hourly_data.tz_convert('Asia/Dhaka')

        time_diffs = hourly_data.index - dt
        closest_idx = time_diffs.abs().argmin()
        closest = hourly_data.iloc[closest_idx]

        fill_map = {
            'temp_min': 'temp',
            'temp_max': 'temp',
            'air_pressure': 'pres',
            'air_humidity': 'rhum',
            'wind_speed': 'wspd',
            'wind_deg': 'wdir',
            'coco': 'coco'
        }
        for col, key in fill_map.items():
            if pd.isna(row[col]) and key in closest:
                val = closest.get(key)
                if pd.notna(val):
                    row[col] = val

        return row
except Exception:
    pass # Silent, don't print per-row errors

# Fallback to daily data
try:
    date_only = pd.Timestamp(dt.date())
    daily = Daily(point, date_only, date_only)
    daily_data = daily.fetch()

    if not daily_data.empty:
        daily_row = daily_data.iloc[0]
        fill_map = {
            'temp_min': 'tmin',
            'temp_max': 'tmax',
            'air_pressure': 'pres',
            'air_humidity': 'rhum',
            'wind_speed': 'wspd',
            'wind_deg': 'wdir',
            'coco': 'coco'
        }
        for col, key in fill_map.items():
            if pd.isna(row[col]) and key in daily_row:
                val = daily_row.get(key)

```

```

        if pd.isna(val):
            row[col] = val
    except Exception:
        pass # Silent, don't print per-row errors

    return row

```

```

tqdm.pandas(desc="Filling Weather Data")
df_lat_lon = df_lat_lon.progress_apply(fill_weather, axis=1)

```

```

Filling Weather Data: 79%|██████████| 1277/1617 [00:30<00:29, 11.61it/s]Warning: Cannot load daily/2019/41895.csv.gz from https://data.meteostat.net/
Filling Weather Data: 100%|██████████| 1617/1617 [00:48<00:00, 33.42it/s]

```

```
missing_data_report(df_lat_lon)
```

```
Group 1: Columns with >70% missing data
```

```

Column Missing %
coco          100.0

```

```
Group 2: Columns with >40% to ≤70% missing data
```

```

Column Missing %
air_humidity  53.061224
feels_like    53.061224
clouds_sky    53.061224
wind_deg      53.061224
weather_main   52.999382
weather_description 52.999382
profession_description 49.412492
financial_condition 45.516388

```

```
Group 3: Columns with >20% to ≤40% missing data
```

```

Column Missing %
workplace      37.909709
air_pressure   30.426716
wind_speed     30.179344
temp_min       29.870130
temp_max       29.560915
marital_status 23.933210

```

```
Group 4: Columns with ≤20% missing data
```

```

Column Missing %
profession_group 18.923933
reason           16.821274
reason_description 16.635745
age              12.739641
time             12.059369
time_hour        12.059369
age_group         7.421150
longitude         6.493506
latitude          6.493506
unix_time         5.009276
method            2.906617
religion          2.411874
suicide_date      2.040816
hometown          1.669759
gender            0.432900

```

```
# # prompt: give me the code make the latest df downloadable and download that db named final_suicide_cleaned.xlsx ins excel format
```

```
# # Make the latest DataFrame downloadable
# from google.colab import files

# # Specify the filename for the Excel file
# filename = '3_weather.xlsx'

# # Save the DataFrame to an Excel file
# df_lat_lon.to_excel(filename, index=False)

# # Download the file
# files.download(filename)
```